

COMP2048: Theory of Computing

Dr. Kasra Khosoussi

Week 8

24/4/2026

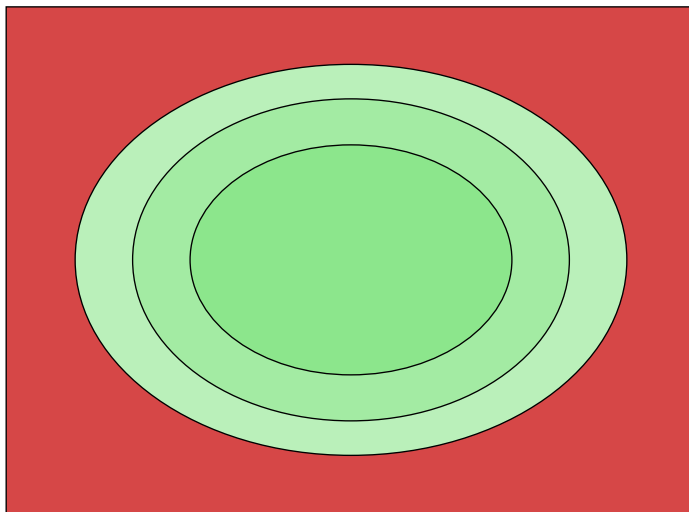
Recap: Computability

Computability Theory

Is language L *decidable*?

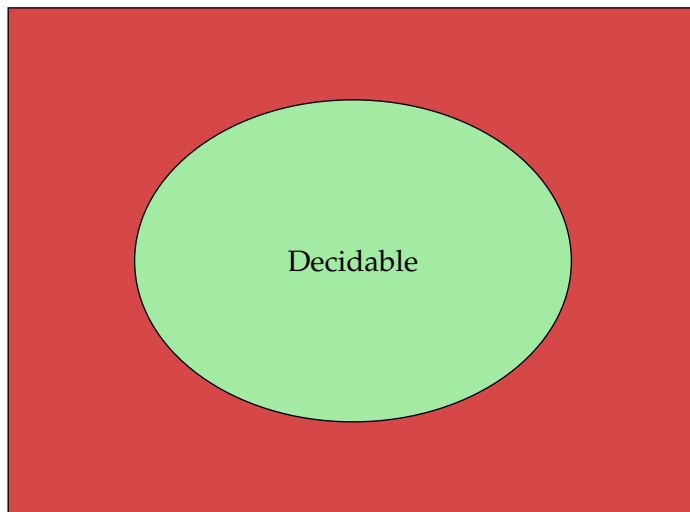
Is there an “algorithm” that solves the corresponding decision problem?

Computability Big Picture



Rectangle: set of all languages (or decision problems) over Σ

Computability Big Picture



Rectangle: set of all languages (or decision problems) over Σ

Decidable $\neq$ Practical

- A problem may be **decidable** but still hopelessly slow to solve
- Suppose a decider uses 2^n steps on inputs of length n
- For $n = 100$:
$$2^{100} \text{ steps}$$
- At 10^9 steps/second $\rightarrow$ far longer than Sun's remaining lifetime!

decidable does not mean **practical**

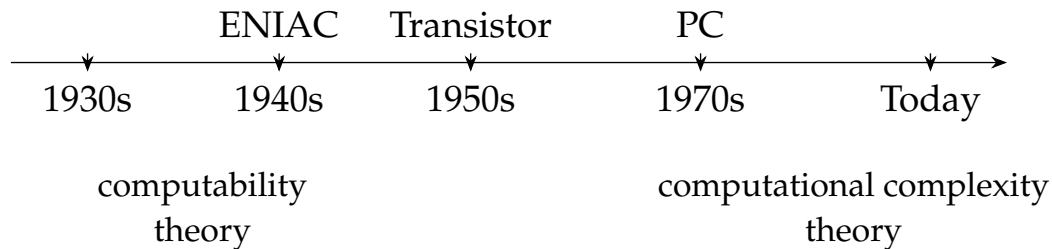
Today's and Next Week's Plan

Complexity Theory

- Can the problem be solved “*efficiently*”?
- Classify decidable problems based on the resources they require
- What problems can be solved with limited resources?
(time, space, etc)

we focus on one basic resource: **time**

Timeline



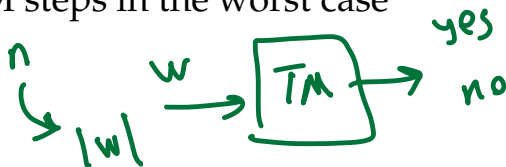
Complexity Theory Questions

- How should we measure the running time of a Turing machine?
- How should we classify languages by their running time?
- What does it mean for an algorithm to be *efficient*?
- Which problems can be solved *efficiently*?

How to measure running time? Time Complexity

Measure running time by

- counting the number of TM steps in the worst case
- as a function input length



Worst-Case Analysis:

- Let M be a deterministic 1-tape TM (decider)
- For input w , let $t(w)$ be number of steps that M takes on input w
- For inputs of length n , define the **worst-case running time**

$f(2) = 5$

Handwritten diagram showing a vertical stack of symbols: two circles at the top, followed by a vertical line, then a circle, then a vertical line, and finally two vertical lines at the bottom.

$$f(n) = \max \{ t(w) \mid |w| = n \}$$

$f(n)$ is max number of steps M may take on any input of length n

Asymptotic Time Complexity

- Exact step counts are usually hard to calculate
- Exact # of steps depends on the model, implementation details, ...
- We care about how time grows as input size grows for large inputs
- *Rate of growth* as a function of input size

Big-O notation

We write $f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0.$$

Examples:

- $3n + 7 = O(n)$ 
- $5n^2 + 2n + 1 = O(n^2)$
- $3n^3 = O(n^3)$
- $n \log n + n = O(n \log n)$
- 2^n is *not* $O(n^k)$ for any fixed k

Ignore:

- lower order terms
- constant factors

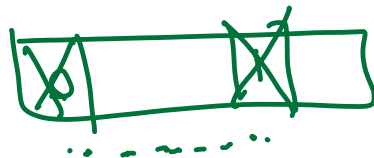
$$O(2^n)$$

Example: estimating TM running time

w

- Consider the language

$$L = \{0^k 1^k \mid k \geq 0\}$$



- first check the input has the form 0^*1^*
 - mark the leftmost unmarked 0
 - scan right and mark the matching unmarked 1
 - return left and repeat
- $O(n)$ steps for the initial check
 - Each round takes $O(n)$ steps
 - There are at most $n/2$ rounds
 - Total running time: $n/2 \times O(n) + O(n) = O(n^2)$

$$\underbrace{O(n^2)} + \cancel{O(n)} = O(n^2)$$

Efficient (Time Complexity) $\triangleq$ Polynomial Time

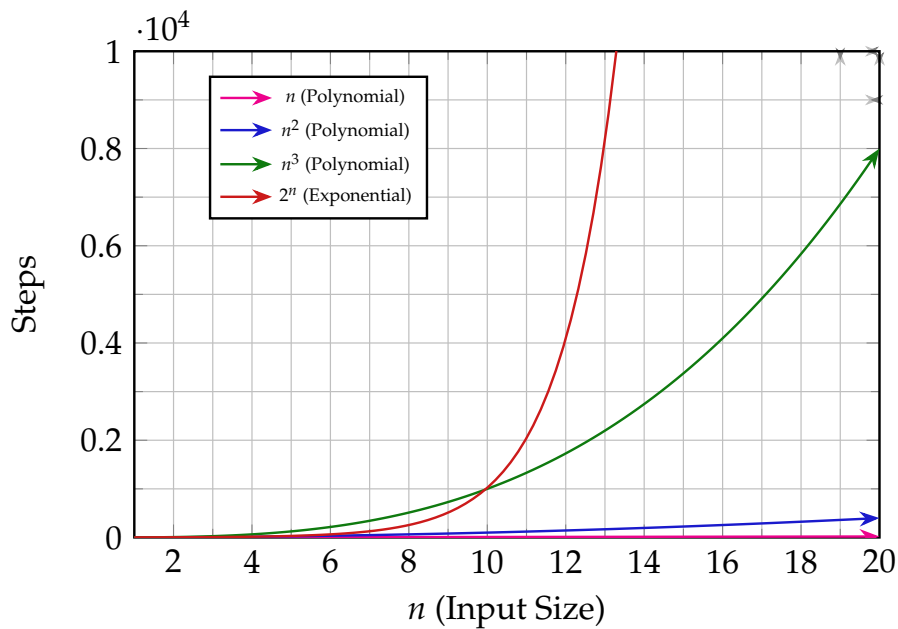
Cobham-Edmonds thesis: polynomial time $\Leftrightarrow$ efficient computation

- Polynomial functions

$$n, n^2, n^3, \dots, n^k$$

grow much more slowly than exponential functions like 2^n

- Polynomial-time computations compose well:
 - polynomial time + polynomial time = polynomial time
 - polynomially many repetitions of polynomial time work is still polynomial



- At $n = 100$, $n^3 = 1,000,000$ while $2^{100} \approx 10^{30}$

Which runtimes count as “efficient”?

- $n^2 - 3n + 10^{10000} = O(n^2)$

- $n \log n$

- $n^{10^{10}}$

- n^n

- $n!$

- 2^n

- 1.00000001^n

- 10^{500}

$$n^k = O(2^n)$$

Time Complexity Class $\text{TIME}(f(n))$

Definition

For a function $f : \mathbb{N} \rightarrow \mathbb{N}$,

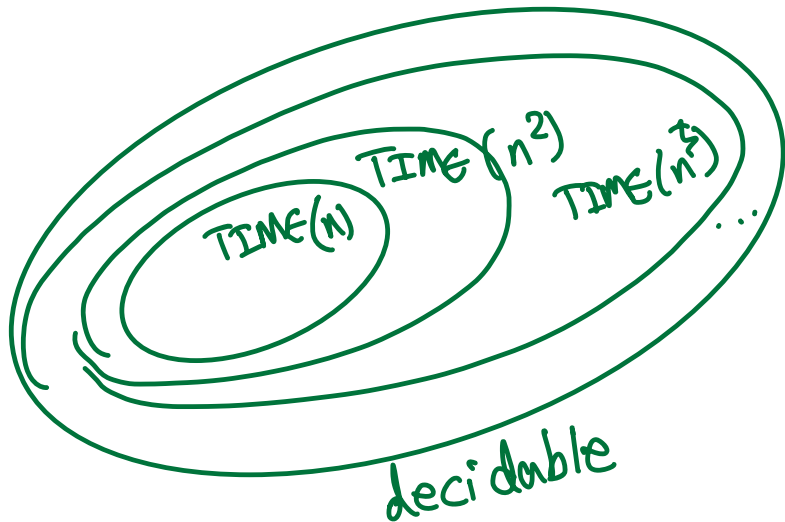
$$\text{TIME}(f(n)) = \{L \mid \text{deterministic TM decides } L \text{ in time } O(f(n))\}.$$

- Input length is n
- Running time means **worst-case** running time
- The machine must halt on every input

Example: we just showed that

$$\{0^k 1^k \mid k \geq 0\} \in \text{TIME}(n^2)$$

$\{0^k 1^k \mid k \geq 0\}$
? $\in \text{TIME}(n)$
 $\{0^k 1^k \mid k \geq 0\}$
 $\text{TIME}(n^2)$



$$\underline{\text{TIME}(n^{100})} \stackrel{?}{\subseteq} \text{TIME}(2^n)$$

Sensitivity of $\text{TIME}(f(n))$ to Model of Computation

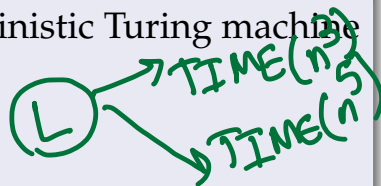
- Unfortunately $\text{TIME}(f(n))$ depends on the model of computation
- Example: for $L = \{0^k 1^k : k \geq 0\}$,
 - with one-tape TMs $L \in \text{TIME}(n^2)$
 - with two-tape TMs $L \in \text{TIME}(n)$
- **But** this dependence is at most **polynomial**:
e.g., any $t(n)$ -time multi-tape TM (when $t(n) \geq n$) can be simulated by a one-tape TM in time $O(t(n)^2)$
- We may only get **polynomial** slowdowns/speedups by switching between reasonable deterministic models of computations

The Class P

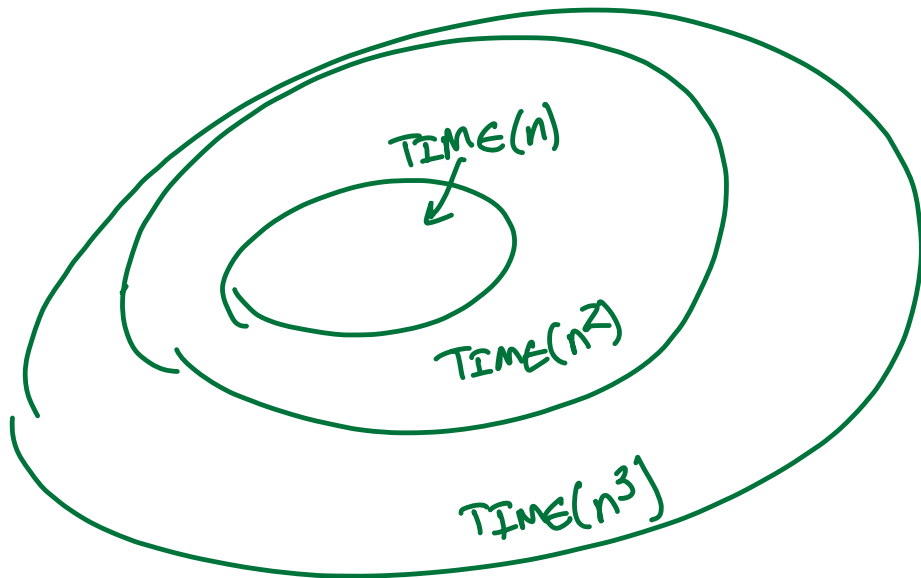
Definition

P is the class of languages decidable by a deterministic Turing machine in polynomial time, i.e.,

$$P = \bigcup_{k \geq 0} \text{TIME}(n^k).$$



- $L \in P$ if some deterministic TM decides L in time $O(n^k)$ for some constant k
- Unlike $\text{TIME}(f(n))$, the class P does not depend on a particular deterministic model of computation
- Intuitively, P is class of “efficiently” solvable decision problems



• language L

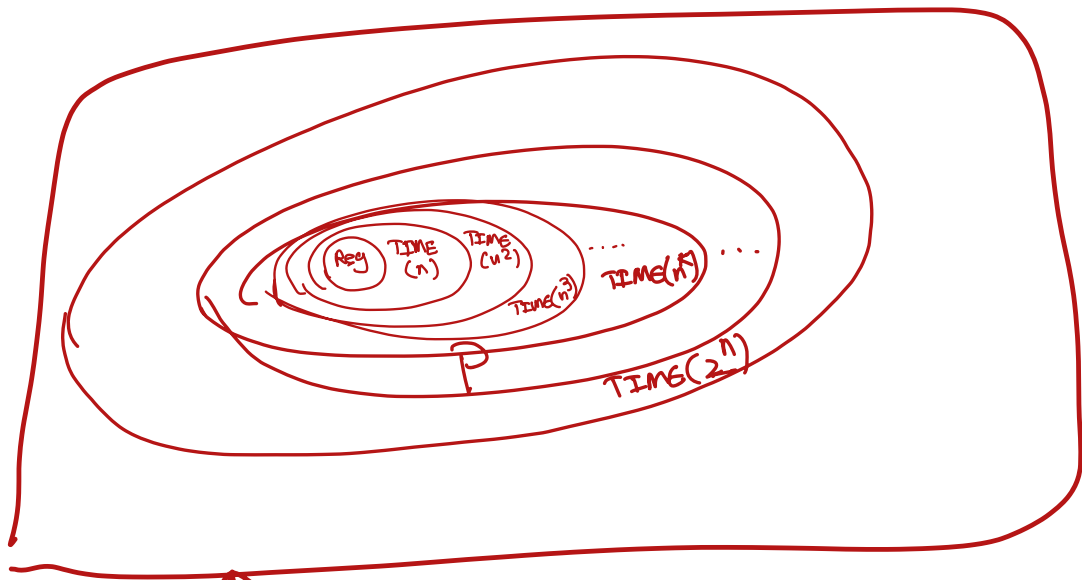
Examples in P

$\in \text{TIME}(n^2) \subseteq P$

- $\{0^k 1^k \mid k \geq 0\} \in P$
- $\{ww^R \mid w \in \{0,1\}^*\} \in P$
- All regular languages are in P (why?)

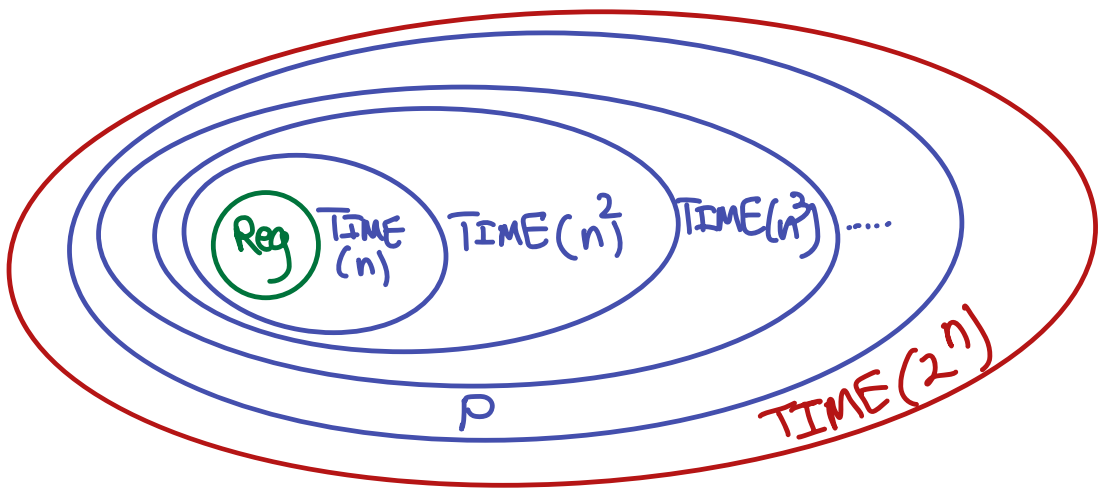
All regular languages are in $\text{TIME}(n)$

A New Big Picture



$$n^k = O(2^n)$$

all decidable languages



all decidable
languages

When checking a solution is easy . . .

- Sometimes finding a solution may be difficult
- But if someone hands us a candidate solution, we can check it quickly
- This leads to the idea of **verification**
- That idea is captured by the class NP
 - P: easy to *solve*
 - NP: easy to *verify*
- We will make this formal next lecture